

Classification in practice

Machine Learning in Finance · Session 9

The classifier so far

0.533

majority rule

0.741

accuracy

0.847

AUC

Last time a logistic regression predicted whether volatility would be higher next month. On the 482 test days it beat the majority class, 74 percent against 53.

Three things were left at their defaults: the threshold of one half, the meaning of the probabilities, and the fact that there were two classes.

The label was an even split

Python Code

```
table = pd.read_csv("data/market_features.csv", parse_dates=["date"]).set_index("date")
table["rising"] = (table["vol_next"] > table["vol_20d"]).astype(int)

print(table["rising"].mean())
```

```
0.4781144781144781
```

The label is 1 on 48 percent of the days, so the two classes are almost the same size. Most decisions a bank or an insurer makes are not like that.

Today

- what a **rare** class does to accuracy
- the **threshold**, set from the cost of each mistake
- **calibration**: whether a probability of 0.3 means 0.3
- a label with **three** classes instead of two
- **k-nearest neighbours**, a second classifier

The lecture introduces each of these. The exercises and the case are where you practise them.

1 • When one class is rare

PART 1

A second table, in which one borrower in five defaults.

The credit table

Python Code

```
credit = pd.read_csv("data/credit.csv")

print(credit.shape)
credit.head(4)
```

(30000, 8)

	limit	age	late_now	months_late	bill	paid	utilisation	default
0	20000	24	2	2	3913	0	0.1956	1
1	120000	26	0	2	2682	0	0.0223	1
2	90000	34	0	0	29239	1518	0.3249	0
3	50000	37	0	0	46990	2000	0.9398	0

Payment records of 30,000 credit card holders at a Taiwanese bank in 2005, from the UCI Machine Learning Repository. One row per borrower, and amounts in New

What the columns hold

column	what it is
limit	the credit limit on the account
age	the borrower's age in years
late_now	months behind on the most recent bill, 0 if not behind
months_late	how many of the last six months were at least one month late
bill	the most recent bill
paid	the amount paid against that bill
utilisation	the most recent bill divided by the credit limit
default	1 if the borrower defaulted the following month

One borrower in five

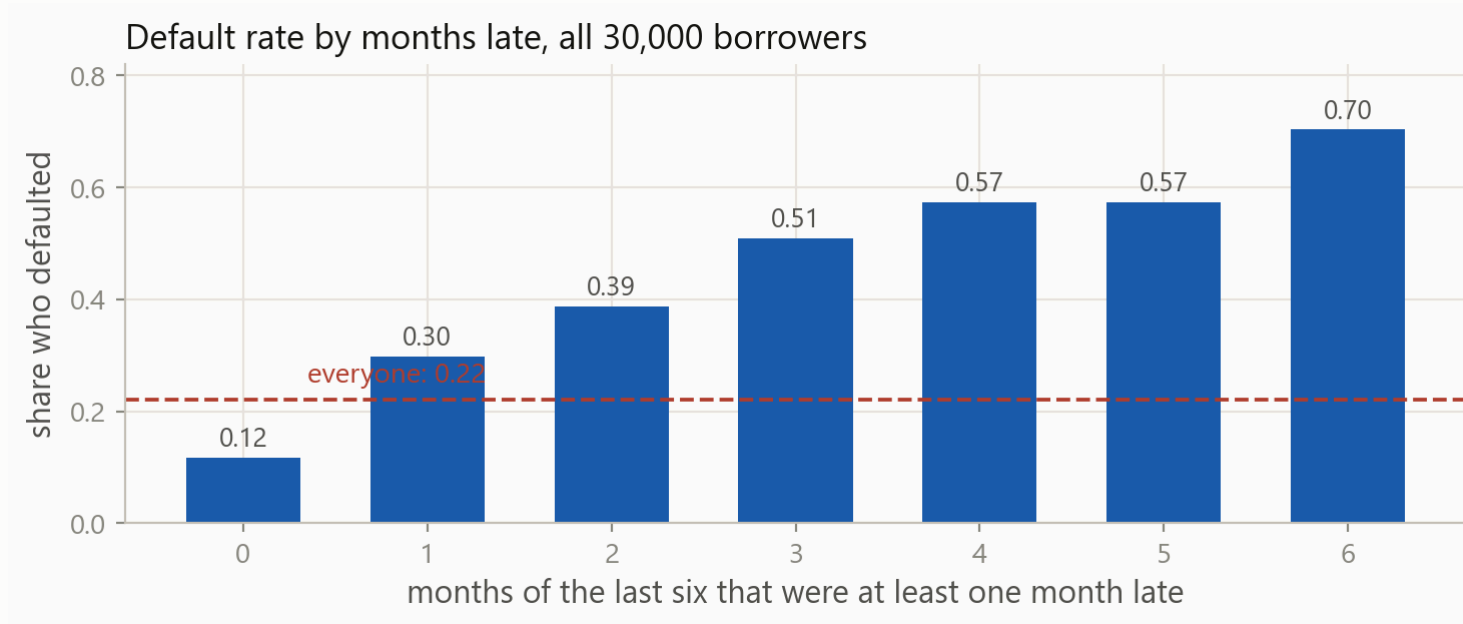
Python Code

```
print(credit["default"].sum())  
print(credit["default"].mean())
```

```
6636  
0.2212
```

6,636 of the 30,000 borrowers defaulted, which is 22 percent. The column already holds ones and zeros, so there is no label to build.

What the columns say about default



A borrower who was never late defaults 12 percent of the time. One who was late in all six months defaults 70 percent of the time.

Borrowers, not days

The market table

Each row is a trading day, and the rows are in time order. A model fitted on 2023 and scored on 2020 would be using the future to predict the past, so the split is made by date and the folds move forward.

The credit table

Each row is a different borrower, and the order they appear in carries no meaning. Any borrower may go in the training rows and any other in the test rows, so the split can be made at random.

`train_test_split` shuffles the rows and cuts them in two. It is the standard tool, and it is wrong for a time series.

Splitting at random

Python Code

```
from sklearn.model_selection import train_test_split

train, test = train_test_split(credit, test_size=0.3, random_state=0,
                               stratify=credit["default"])

print(len(train), len(test))
print(round(test["default"].mean(), 4))
```

```
21000 9000
0.2212
```

random_state=0 fixes which borrowers go where, so the split is the same on every run. **stratify** keeps the share of defaults equal in both halves.

The same four lines

Python Code

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

columns = ["limit", "age", "late_now", "months_late", "bill", "paid", "utilisation"]
model = Pipeline([("scale", StandardScaler()), ("logit", LogisticRegression())])
model.fit(train[columns], train["default"])
p = model.predict_proba(test[columns])[:, 1]
```

Nothing about the model changed. The columns are in different units, so the scaler goes in front of the classifier, as it did for ridge.

Accuracy against the majority rule

Python Code

```
from sklearn.metrics import accuracy_score, roc_auc_score

predicted = model.predict(test[columns])

print(round(accuracy_score(test["default"], predicted), 4))
print(round(1 - test["default"].mean(), 4))
```

0.815

0.7788

The model is right about 81.5 percent of the 9,000 test borrowers. Predicting that nobody defaults is right 77.9 percent of the time.

What the model predicted

Python Code

```
from sklearn.metrics import confusion_matrix

print(confusion_matrix(test["default"], predicted))
print(predicted.sum())
```

```
[[6715  294]
 [1371  620]]
914
```

Rows are what happened, columns are the prediction. The model called 914 defaults and was right about 620; 1,991 borrowers defaulted, so it missed 1,371.

Precision and recall, on a rare class

Python Code

```
from sklearn.metrics import precision_score, recall_score

print(round(precision_score(test["default"], predicted), 3))
print(round(recall_score(test["default"], predicted), 3))
```

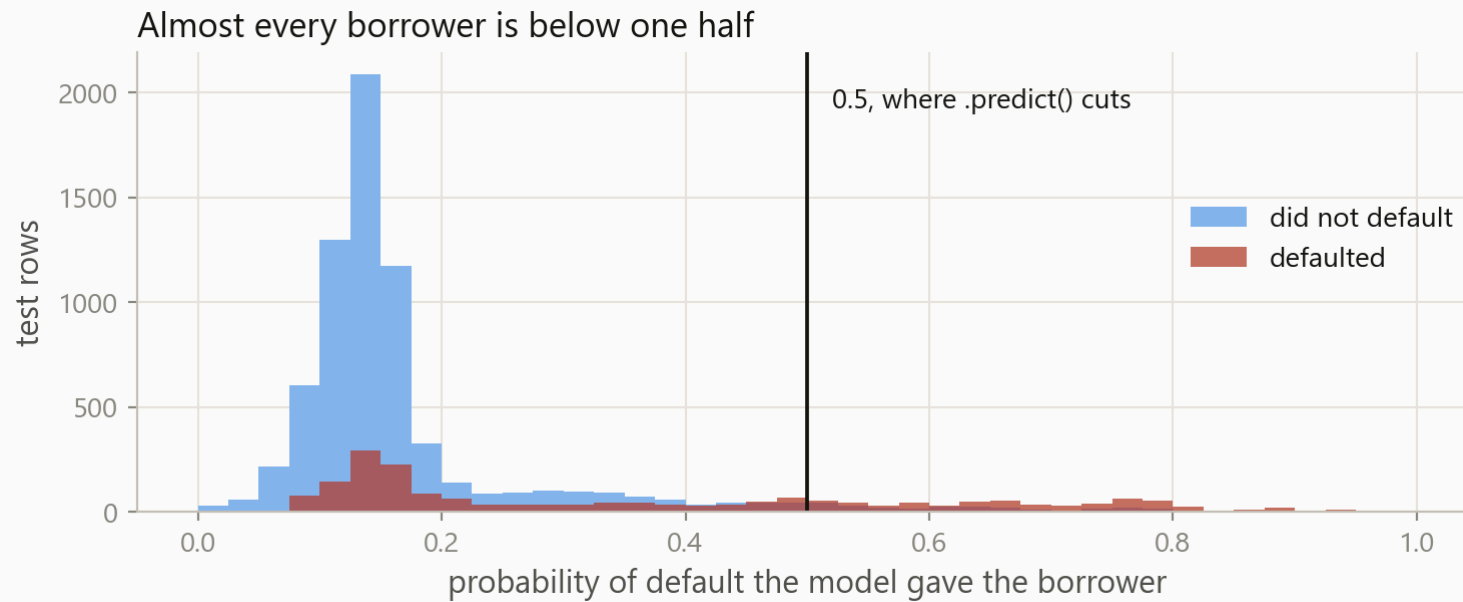
0.678

0.311

Of the borrowers the model called a default, 68 percent defaulted; of those who defaulted, it caught 31 percent.

When one class is rare, report recall beside accuracy. The accuracy of 0.815 is made of the borrowers the model correctly left alone.

The probabilities the model gave



Only 914 of the 9,000 borrowers reach 0.5, so `.predict()` calls almost everyone safe. The threshold produced the low recall, not the model.

Your turn 1

Predict a default for every test borrower whose probability reaches 0.2, and report the recall and the precision. Decide first which way each will move.

Python Code

```
lower = ...

print(round(recall_score(test["default"], lower), 3))
print(round(precision_score(test["default"], lower), 3))
```

`p` holds the probability of default for each test borrower, from `predict_proba` two slides ago. Compare it with 0.2 and turn the answer into ones and zeros, as the label itself was made.

Your turn 1: answer

```
lower = (p >= 0.2).astype(int)
```

Recall rises from 0.311 to **0.574** and precision falls from 0.678 to **0.486**. The model called 2,353 defaults instead of 914, and caught 1,143 of the 1,991 instead of 620. The model itself never changed.

A missed default is not a false alarm

Missing a default

The bank lends, the borrower does not repay, and the balance is written off. The loss is the size of the credit extended.

A false alarm

The bank refuses or reduces a limit for a borrower who would have paid. The loss is the interest that borrower would have earned, and the customer.

The two mistakes are not worth the same, so the threshold with the fewest mistakes is not the cheapest one. Suppose a missed default costs 5 and a false alarm costs 1.

Counting the cost

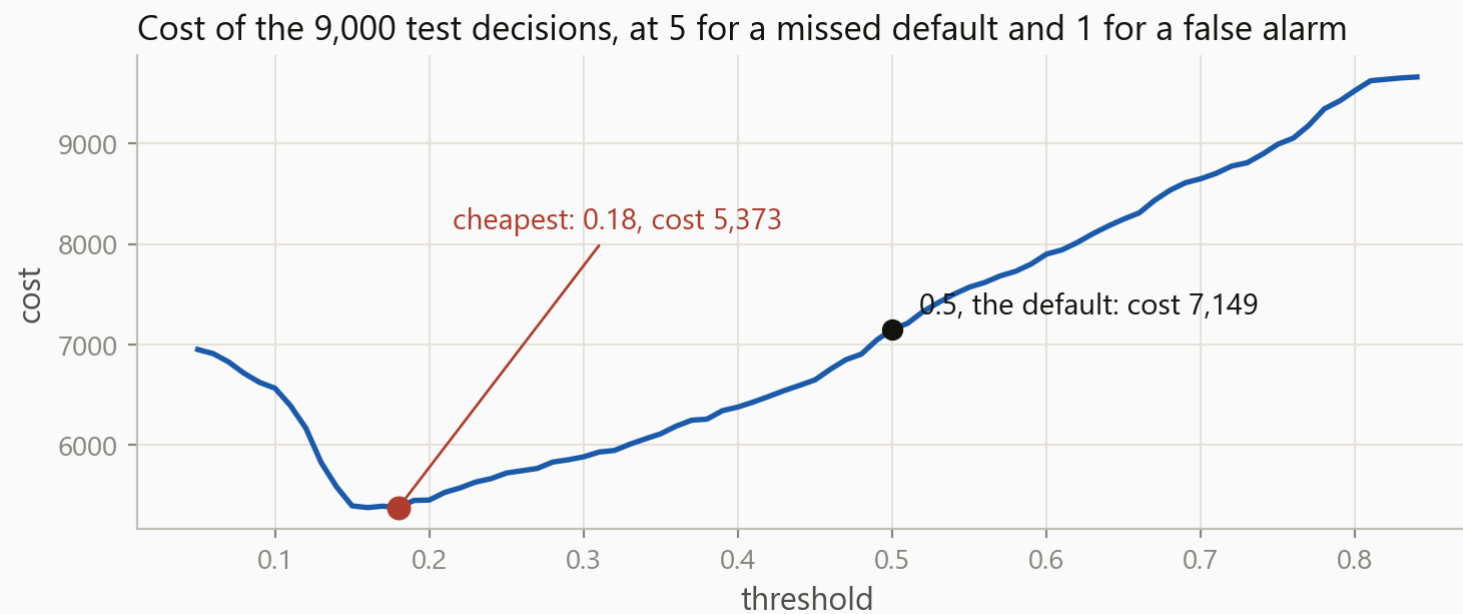
Python Code

```
for threshold in [0.1, 0.2, 0.3, 0.5]:  
    called = (p >= threshold).astype(int)  
    tn, fp, fn, tp = confusion_matrix(test["default"], called).ravel()  
    print(threshold, fn, fp, 5 * fn + fp)
```

```
0.1 93 6099 6564  
0.2 848 1210 5450  
0.3 1019 785 5880  
0.5 1371 294 7149
```

`.ravel()` flattens the 2 by 2 array into its four counts, in reading order. The last number is the cost of the 9,000 decisions: 7,149 at one half, 5,450 at 0.2.

The cheapest threshold



Sweeping the threshold in steps of 0.01 puts the minimum at 0.18, where the 9,000 decisions cost 5,373 instead of 7,149. One number, a quarter of the cost.

The threshold from the costs

$$\text{threshold} = \frac{\text{cost of a false alarm}}{\text{cost of a false alarm} + \text{cost of a miss}} = \frac{1}{1 + 5} = 0.167$$

Predicting a default pays when the probability times 5 is larger than 1, which is a probability above one sixth. The sweep found 0.18, and the two thresholds cost 5,373 and 5,386.

Choosing the threshold on training rows

Python Code

```
p_train = model.predict_proba(train[columns])[:, 1]
for threshold in [0.14, 0.16, 0.18, 0.20]:
    called = (p_train >= threshold).astype(int)
    tn, fp, fn, tp = confusion_matrix(train["default"], called).ravel()
    print(threshold, 5 * fn + fp)
```

```
0.14 12628
0.16 11991
0.18 12178
0.2 12482
```

The same sweep on the 21,000 training rows picks 0.16, against the 0.18 the test rows would pick. The threshold is a setting like **C**, chosen on rows the model may see.

Weighting the rare class

Python Code

```
balanced = Pipeline([("scale", StandardScaler()),  
                     ("logit", LogisticRegression(class_weight="balanced"))])  
balanced.fit(train[columns], train["default"])  
pred_b = balanced.predict(test[columns])  
  
print(round(accuracy_score(test["default"], pred_b), 4))  
print(round(recall_score(test["default"], pred_b), 3))
```

0.7779

0.55

`class_weight="balanced"` counts each default 2.26 times during fitting and each repayment 0.64. Recall rises to 0.550 and accuracy falls to 0.778, below the 0.779 of predicting that nobody defaults.

What the weights changed

Python Code

```
p_balanced = balanced.predict_proba(test[columns])[:, 1]

print(round(roc_auc_score(test["default"], p), 4))
print(round(roc_auc_score(test["default"], p_balanced), 4))
```

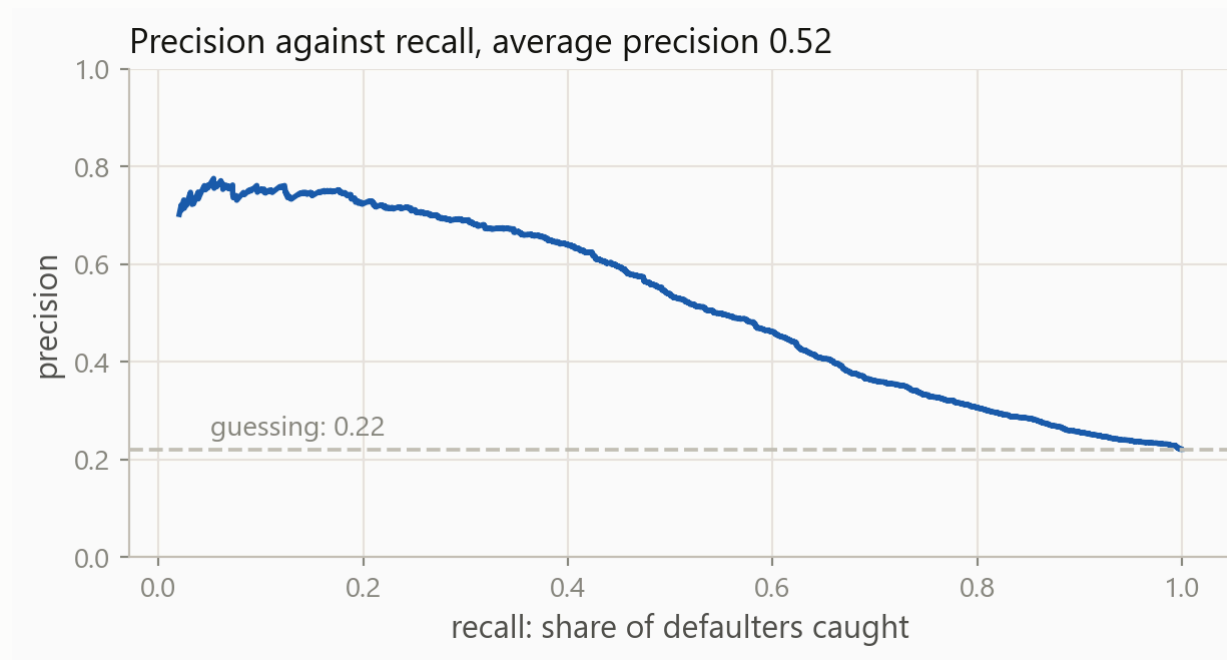
0.7476

0.7481

The weights moved where the cut falls, not the ranking of the borrowers.

At a threshold of 0.2 the plain model reaches recall 0.574 at precision 0.486, and the weighted model reaches 0.550 at 0.498: the same place on the same curve. Only the threshold can be tuned to a cost.

Precision against recall



Every threshold is one point on this curve, and **average_precision_score** summarises it: 0.52 against 0.22 for guessing. On a rare class it says more than the ROC curve, which is dominated by true negatives.

2 · Do the probabilities mean what they say?

PART 2

Whether a model that says 0.30 is right about three borrowers in ten.

The question

Take every borrower the model gave about 0.30, and count how many defaulted.

About 30 percent

the number is a probability, and multiplying it by an amount of money gives an expected loss

12 percent

the ranking may still be useful, but any decision that compares the number with a cost will be wrong

A model that passes this check is called calibrated.

Bucketing the test rows

Python Code

```
test = test.copy()
test["p"] = p
test["bucket"] = pd.cut(test["p"], [0, 0.2, 0.4, 0.6, 1.0])

print(test.groupby("bucket", observed=True)["default"].agg(["size", "mean"]).round(3))
```

	size	mean
bucket		
(0.0, 0.2]	6647	0.128
(0.2, 0.4]	1084	0.309
(0.4, 0.6]	643	0.561
(0.6, 1.0]	626	0.714

The plain model, bucket by bucket

the model said	borrowers	share that defaulted
0.0 to 0.2	6,647	0.128
0.2 to 0.4	1,084	0.309
0.4 to 0.6	643	0.561
0.6 to 1.0	626	0.714

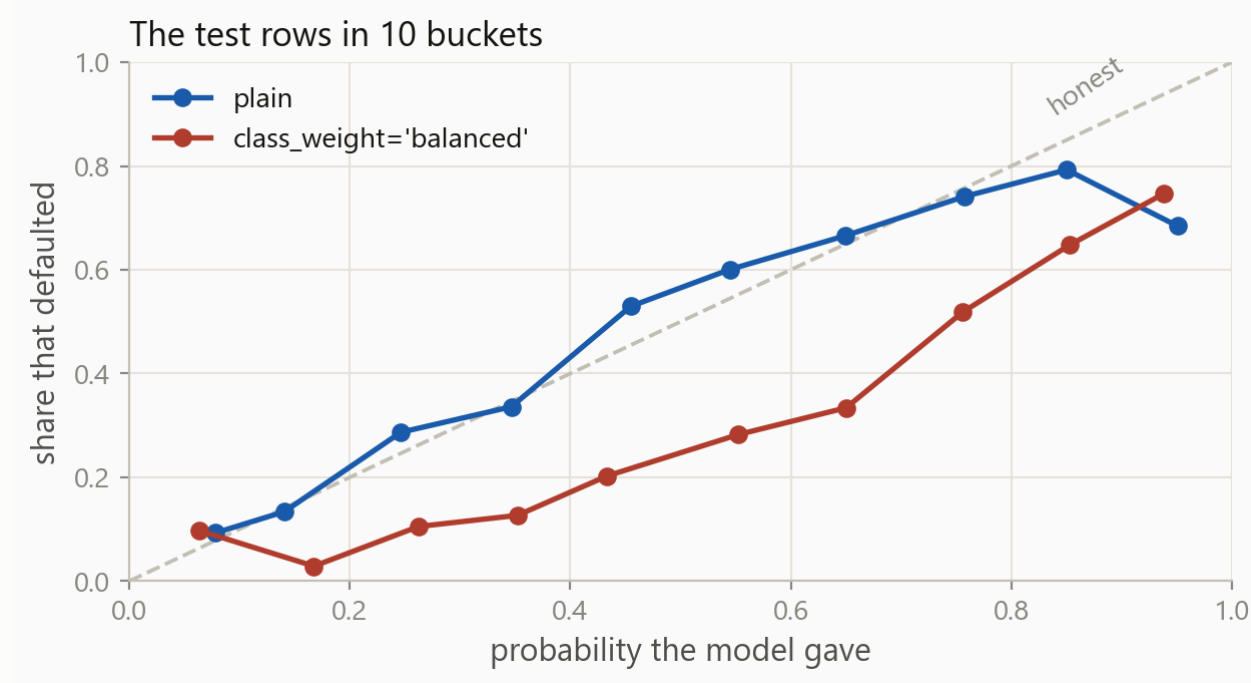
Every bucket’s share falls inside its own range, and the average probability is 0.2187 against the 0.2212 that defaulted.

The balanced model, the same borrowers

the model said	borrowers	share that defaulted
0.0 to 0.2	214	0.042
0.2 to 0.4	5,518	0.121
0.4 to 0.6	1,502	0.226
0.6 to 1.0	1,766	0.551

Of the 5,518 borrowers it put between 0.2 and 0.4, 12 percent defaulted. The average probability is 0.439 against the same 0.2212, so every bucket says more than happens.

Said against happened



A calibrated model sits on the dashed diagonal. The plain model follows it closely; the weighted model sits below it everywhere.

Why the weights did that

The weights make the training rows behave as though 50 percent of borrowers default instead of 22 percent. The probabilities are then right for that invented population, not for the real one.

Anything that changes the mix of the training rows changes the level of the probabilities: class weights, dropping rows of the common class, or copying rows of the rare one.

The Brier score

Python Code

```
from sklearn.metrics import brier_score_loss

print(round(brier_score_loss(test["default"], p), 4))
print(round(brier_score_loss(test["default"], p_balanced), 4))
```

0.1395

0.1879

The Brier score is the mean squared error of the probabilities, with what happened as 1 or 0. It is 0.1395 for the plain model and 0.1879 for the weighted one, and smaller is better.

Ranking against level

What AUC measures

Whether the borrowers who defaulted were given higher numbers than those who did not. Only the order matters, so 0.7476 and 0.7481 say the two models rank almost identically.

What calibration measures

Whether the numbers are right in level. The Brier score separates the two models, 0.1395 against 0.1879, because one of them says 0.44 on average where 0.22 happens.

A model can rank perfectly and still be useless for a decision made by comparing a probability with a cost.

Recalibrating the weighted model

Python Code

```
from sklearn.calibration import CalibratedClassifierCV

fixed = CalibratedClassifierCV(balanced, method="sigmoid", cv=5)
fixed.fit(train[columns], train["default"])
p_fixed = fixed.predict_proba(test[columns])[:, 1]

print(round(p_fixed.mean(), 4))
print(round(brier_score_loss(test["default"], p_fixed), 4))
```

0.2186

0.1395

CalibratedClassifierCV learns a second, small model that maps the first model's numbers onto probabilities, on folds. The average comes back to 0.2186 and the Brier score to 0.1395.

The cost threshold on each model

Python Code

```
for name, prob in [("plain", p), ("balanced", p_balanced), ("fixed", p_fixed)]:  
    called = (prob >= 1 / 6).astype(int)  
    tn, fp, fn, tp = confusion_matrix(test["default"], called).ravel()  
    print(name, 5 * fn + fp)
```

```
plain 5386  
balanced 6940  
fixed 5385
```

One sixth costs 5,386 on the plain model and 6,940 on the weighted one, where 0.167 does not mean the same thing. Calibrating brings it back to 5,385.

3 · More than two classes

PART 3

A label with three values instead of two.

Three answers instead of two

Whether next month is more volatile is a coarse question: a rise of one percent and a rise of 60 percent both count as a rise.

Python Code

```
ratio = table["vol_next"] / table["vol_20d"]
```

```
print(round(ratio.median(), 3))
```

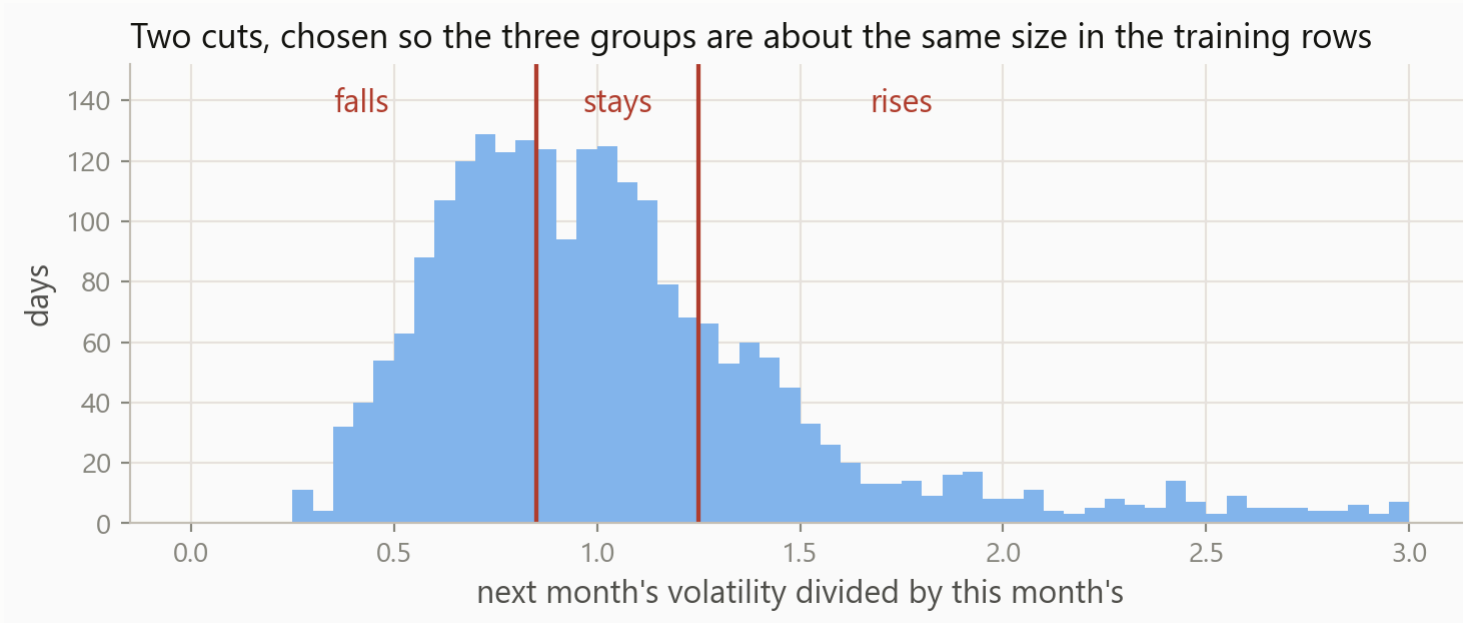
```
print(round(ratio.max(), 2))
```

0.978

6.07

The ratio of next month's volatility to this month's is 0.978 in the middle and 6.07 at its largest. Its right tail is long, so the two cuts are not symmetric around one.

Two cuts



Below 0.85 next month is at least 15 percent calmer and above 1.25 at least 25 percent busier. The two cuts split the training days 38, 32 and 29 percent.

Three labels in one line

Python Code

```
table["move"] = pd.cut(ratio, [-np.inf, 0.85, 1.25, np.inf],  
                        labels=["falls", "stays", "rises"])  
  
print(table["move"].value_counts())
```

```
move  
falls      898  
stays      834  
rises      644  
Name: count, dtype: int64
```

`pd.cut` is the function that bucketed the probabilities, with names given to the ranges. The label is text rather than a number, which scikit-learn accepts.

The majority rule, with three classes

Python Code

```
train = table.loc[:"2022-12-31"]
test = table.loc["2023-01-01":]

print(train["move"].value_counts(normalize=True).round(3))
print(round((test["move"] == "falls").mean(), 4))
```

```
move
falls    0.385
stays    0.323
rises    0.292
Name: proportion, dtype: float64
0.3506
```

Three classes, the same four lines

Python Code

```
three = Pipeline([("scale", StandardScaler()),  
                  ("logit", LogisticRegression(max_iter=1000))])  
three.fit(train[["vol_20d", "ret_20d"]], train["move"])  
  
print(round(accuracy_score(test["move"], three.predict(test[["vol_20d", "ret_20d"]]])), 4))
```

0.5539

Nothing in the call changed. **LogisticRegression** saw three values in the label and fitted three classes, right on 55 percent of the test days against 35 for the majority rule.

classes_ is in alphabetical order

Python Code

```
logit = three.named_steps["logit"]  
  
print(logit.classes_)
```

```
['falls' 'rises' 'stays']
```

The labels come back sorted: **falls**, **rises**, **stays**, not the order the cuts were written in. Every array the model produces uses this order.

One row of coefficients per class

Python Code

```
print(logit.coef_.shape)
print(logit.intercept_.shape)
print(logit.coef_.round(3))
```

```
(3, 2)
(3,)
[[ 0.835  0.246]
 [-0.892 -0.131]
 [ 0.057 -0.114]]
```

With two classes `coef_` had one row. With three it has three, one per class, so the model turns a day's two columns into three numbers instead of one.

Three scores for one day

Python Code

```
day = test[["vol_20d", "ret_20d"]].head(1)

print(logit.classes_)
print(three.decision_function(day).round(3))
```

```
['falls' 'rises' 'stays']
[[ 0.06  -0.324  0.264]]
```

The first test day scores 0.060 for **falls**, -0.324 for **rises** and 0.264 for **stays**. These are not probabilities: one is negative, and the three do not add up to anything in particular.

Making the scores into probabilities

Python Code

```
scores = three.decision_function(day)[0]

print(np.exp(scores).round(3))
print(np.exp(scores).sum().round(3))
print((np.exp(scores) / np.exp(scores).sum()).round(3))
```

```
[1.061 0.724 1.302]
3.087
[0.344 0.234 0.422]
```

The exponential makes every score positive: 1.061, 0.724 and 1.302. Dividing each by their total of 3.087 makes them add to one, giving 0.344, 0.234 and 0.422.

The softmax, for any number of classes

$$P(\text{class } j) = \frac{e^{s_j}}{e^{s_1} + e^{s_2} + \dots + e^{s_N}}$$

With N classes the model produces N scores and this returns N probabilities. It is called the softmax, and with two classes it is the sigmoid from last time.

predict_proba and predict

Python Code

```
probabilities = three.predict_proba(test[["vol_20d", "ret_20d"]])

print(probabilities.shape)
print(probabilities.sum(axis=1)[:4].round(6))
print(three.predict(day))
```

```
(482, 3)
[1.  1.  1.  1.]
['stays']
```

One row per day, one column per class in the order of `classes_`, and every row adds to one. `.predict()` returns the largest of the three.

The confusion matrix is three by three

Python Code

```
order = ["falls", "stays", "rises"]  
predicted = three.predict(test[["vol_20d", "ret_20d"]])  
  
print(confusion_matrix(test["move"], predicted, labels=order))
```

```
[[124  30  15]  
 [ 77  65  80]  
 [  2  11  78]]
```

`labels=` puts the rows and columns in the order that reads naturally. Rows are what happened, columns the prediction, so the diagonal counts the days called correctly: 124, 65 and 78 of 482.

Where the mistakes fall

	said falls	said stays	said rises
was falls	124	30	15
was stays	77	65	80
was rises	2	11	78

A day that fell was called a rise 15 times, and a day that rose was called a fall twice. Only 17 of the 482 days are wrong by two steps.

One recall per class

Python Code

```
from sklearn.metrics import classification_report

print(classification_report(test["move"], predicted, labels=order, digits=3))
```

	precision	recall	f1-score	support
falls	0.611	0.734	0.667	169
stays	0.613	0.293	0.396	222
rises	0.451	0.857	0.591	91
accuracy			0.554	482
macro avg	0.558	0.628	0.551	482
weighted avg	0.582	0.554	0.528	482

Your turn 2

The report prints a macro average of 0.551. Build it yourself from the three per-class F1 scores, then say whether weighting by class size would give something larger or smaller.

Python Code

```
from sklearn.metrics import f1_score

per_class = f1_score(test["move"], predicted, average=None, labels=order)
print(per_class.round(3))
print(round(..., 3))
```

`average=None` returns one F1 per class instead of a single number, in the order given by `labels`.

Your turn 2: answer

```
print(round(per_class.mean(), 3))
```

The three F1 scores are 0.667, 0.396 and 0.591, so the macro average is **0.551**. Weighting is **smaller** at 0.528, because **stays** has 222 of the 482 test days and the worst F1 of the three.

Macro treats a rare class as seriously as a common one. Read it when the rare class is the one that matters.

All 19 columns

Python Code

```
columns_19 = list(table.columns[:19])
wide = Pipeline([("scale", StandardScaler()),
                  ("logit", LogisticRegression(max_iter=1000))])
wide.fit(train[columns_19], train["move"])

print(round(accuracy_score(test["move"], wide.predict(test[columns_19])), 4))
print(wide.named_steps["logit"].coef_.shape)
```

```
0.5332
(3, 19)
```

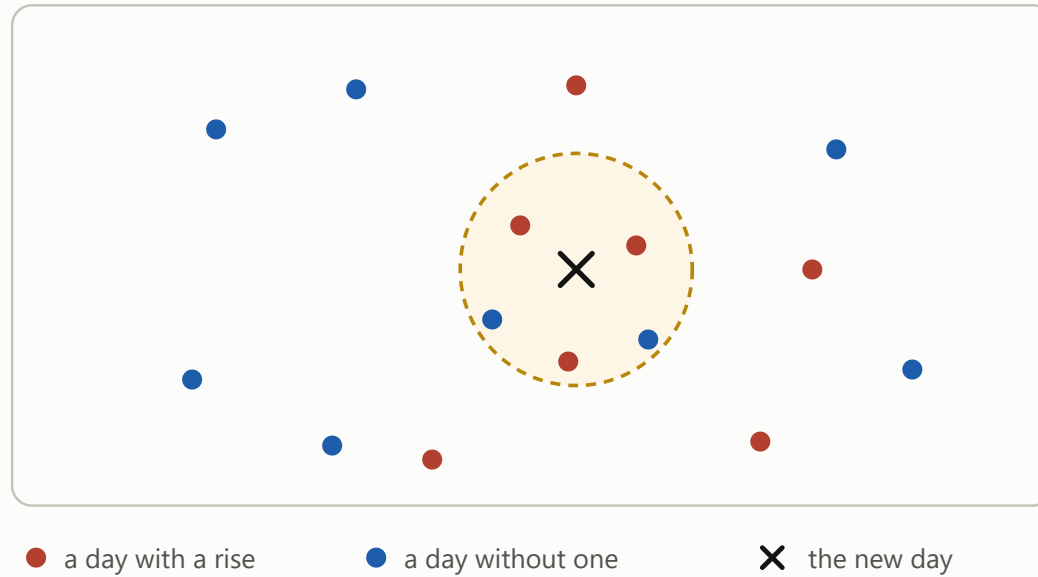
`coef_` is now 3 by 19, which is 57 numbers. The accuracy is 0.533 against 0.554 for two columns, so the extra 17 columns did not help here either.

4 • Another kind of classifier

PART 4

A classifier with no coefficients, put against logistic regression.

The idea



A new day arrives

Measure the distance to every past day and keep the k closest. Here k is 5.

The neighbours vote

3 of the 5 had a rise, so the probability of a rise is 3 of 5, which is 0.6, and the prediction is the class with most votes.

Nothing is fitted, and there are no coefficients.

k-nearest neighbours in code

Python Code

```
from sklearn.neighbors import KNeighborsClassifier

neighbours = Pipeline([("scale", StandardScaler()),
                        ("knn", KNeighborsClassifier(n_neighbors=15))])
neighbours.fit(train[["vol_20d", "ret_20d"]], train["rising"])
p_knn = neighbours.predict_proba(test[["vol_20d", "ret_20d"]])[:, 1]

print(round(roc_auc_score(test["rising"], p_knn), 3))
```

0.634

The four lines are the usual ones, with a different class on the first. Back on the two-class label, k of 15 scores 0.634 against 0.847 for logistic regression.

Distance needs comparable columns

Python Code

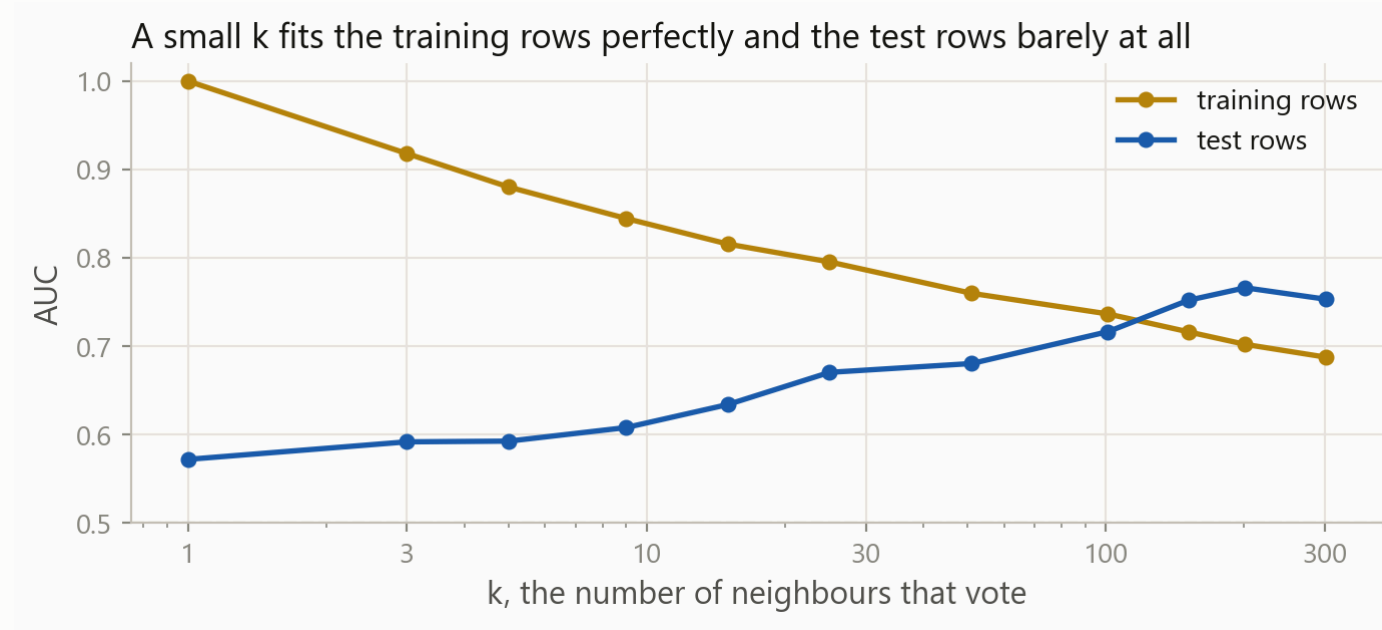
```
c_train, c_test = train_test_split(credit, test_size=0.3, random_state=0,
                                   stratify=credit["default"])
bare = KNeighborsClassifier(n_neighbors=25)
scaled = Pipeline([("scale", StandardScaler()), ("knn", KNeighborsClassifier(n_neighbors=25))])
for candidate in [bare, scaled]:
    candidate.fit(c_train[columns], c_train["default"])
    print(round(roc_auc_score(c_test["default"], candidate.predict_proba(c_test[columns]))[:, 1]), 3))
```

0.627

0.749

On the credit table the AUC is 0.627 without a scaler and 0.749 with one. **limit** varies by about 129,000 and **late_now** by 0.76, so distance means the difference in credit limits.

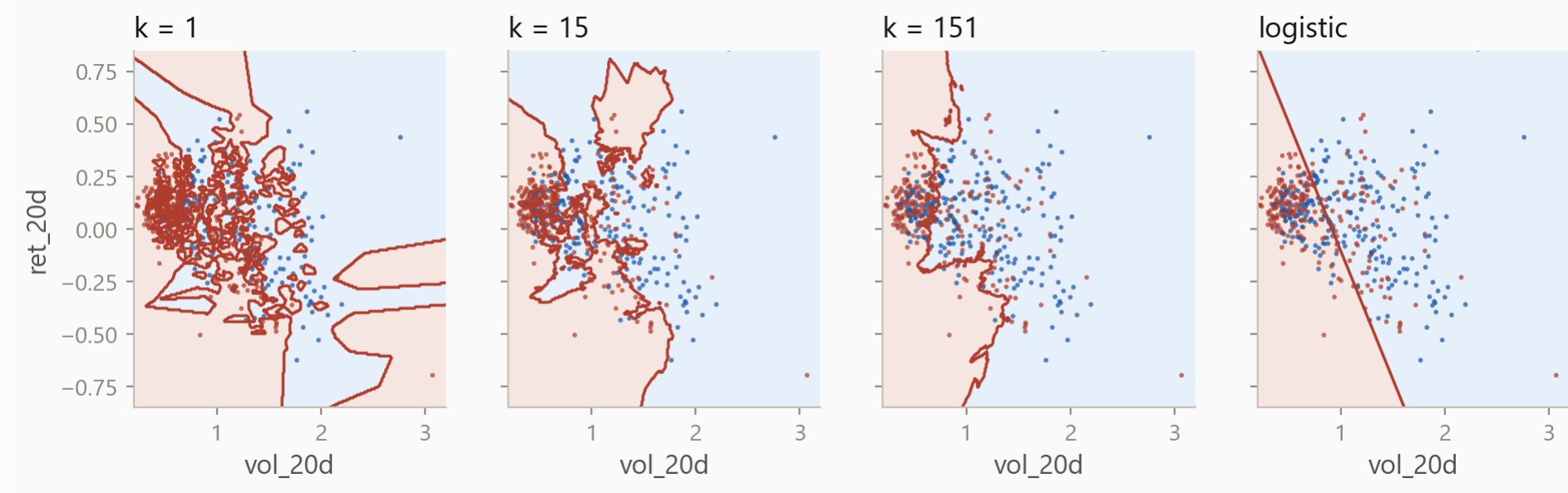
Training and test AUC against k



At k of 1 every training day is its own nearest neighbour, so the training AUC is 1 and the test AUC is 0.572. By k of 151 they are 0.716 and 0.752.

A large k is a simple model, in the same way that a small C was.

What the boundary looks like



Each panel shades where the model gives a probability above one half. At k of 1 the regions are islands around individual days; as k grows they join up.

Choosing k on the folds

Python Code

```
from sklearn.model_selection import GridSearchCV, TimeSeriesSplit

folds = TimeSeriesSplit(n_splits=5)
grid = {"knn__n_neighbors": [1, 5, 15, 51, 101, 151, 201, 301]}
search = GridSearchCV(neighbours, grid, cv=folds, scoring="roc_auc")
search.fit(train[["vol_20d", "ret_20d"]], train["rising"])

print(search.best_params_, round(search.best_score_, 3))
```

```
{'knn__n_neighbors': 301} 0.717
```

The search is the one used for **alpha** and for **C**, with a different setting named. It picks k of 301, with a mean AUC of 0.717 over the five folds.

k cannot exceed the smallest fold

Python Code

```
for fit_rows, score_rows in folds.split(train):  
    print(len(fit_rows), len(score_rows))
```

```
319 315  
634 315  
949 315  
1264 315  
1579 315
```

The first fold fits on 319 days, so any k above 319 asks for more neighbours than there are rows. **GridSearchCV** returns **nan** and warns, which is why the grid stops at

Against logistic regression, same folds

Python Code

```
from sklearn.model_selection import cross_val_score

logistic = Pipeline([("scale", StandardScaler()), ("logit", LogisticRegression())])
scores = cross_val_score(logistic, train[["vol_20d", "ret_20d"]], train["rising"],
                        cv=folds, scoring="roc_auc")

print(round(scores.mean(), 3))
```

0.745

Logistic regression scores 0.745 against 0.717 for the best k, so the folds choose logistic regression. On the test days it is 0.847 against 0.753.

k-NN with three classes

Python Code

```
three_nn = Pipeline([("scale", StandardScaler()),  
                     ("knn", KNeighborsClassifier(n_neighbors=151))])  
three_nn.fit(train[["vol_20d", "ret_20d"]], train["move"])  
  
print(round(accuracy_score(test["move"], three_nn.predict(test[["vol_20d", "ret_20d"]])), 3))  
print(three_nn.predict_proba(test[["vol_20d", "ret_20d"]])[0].round(3))
```

```
0.494  
[0.351 0.232 0.417]
```

A vote works for any number of classes, so the same call takes the three-class label and returns three columns. It is right on 49 percent of the test days against 55 for logistic regression.

Fitting time against prediction time

Logistic regression

Fitting searches for the coefficients and takes the time. Predicting is one multiplication per column, so it is immediate, and the fitted model is a handful of numbers you can read.

k-nearest neighbours

Fitting only stores the training rows. Predicting measures the distance from each new row to all of them, so the cost falls at prediction time and the whole training table has to be kept.

On the 19-column table here: logistic regression fits in 9 milliseconds and predicts in 1, k-NN fits in 2 and predicts in 21.

Other classifiers

Trees and ensembles

split the table one column at a time. They come next, and they need no scaler.

Naive Bayes

treats every column as independent given the class. Fast on wide tables, and false on market columns.

Discriminant analysis

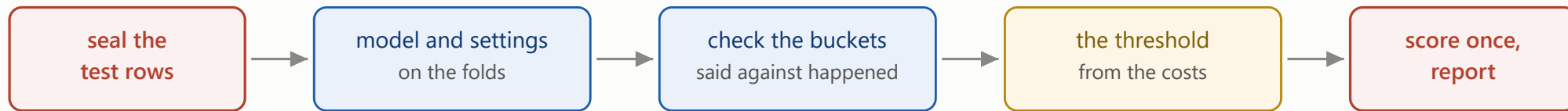
models each class as a bell shape. Close to logistic regression, with more assumptions about the columns.

Support vector machines

choose the boundary with the widest margin. Strong on small tables, and no probability without extra work.

Each gives a probability, and each is scored the same way.

The workflow



everything between the two brick boxes uses training rows only

What stays with you

Accuracy hides a rare class

77.9 percent of borrowers did not default, so predicting that nobody defaults scores 0.779. Report recall beside accuracy, and read the confusion matrix.

Rows that are not a time series

`train_test_split(..., stratify=y, random_state=0)` for a cross-section, `TimeSeriesSplit` for days. `stratify` keeps the rare class in both halves.

Calibration: said against happened

Bucket with `pd.cut` and compare each bucket with what happened. `brier_score_loss` is the one number, `CalibratedClassifierCV` the repair.

Macro or weighted

Macro treats the classes equally, weighted by their size. When the class you care about is the small one, read macro.

The threshold is a decision

`.predict()` uses one half. At a cost of 5 for a miss and 1 for a false alarm, one sixth is cheapest, and saved a quarter of the cost.

class_weight moves the cut, not the ranking

`class_weight="balanced"` raised recall from 0.311 to 0.550 and left AUC at 0.748. It also broke the probabilities.

More than two classes

`classes_` is alphabetical, `coef_` gets a row per class and `predict_proba` a column. The softmax turns the scores into probabilities.

k-nearest neighbours

`KNeighborsClassifier(n_neighbors=k)`, scaled, with k on the folds. A large k is a simple model. Logistic regression won here.

Where this goes next

before	a label as the target: probabilities, predicted labels, and the scores that judge them	logistic regression
now	rare classes, the threshold as a cost decision, calibrated probabilities, three classes, and a second classifier	k-nearest neighbours
next	the whole classification workflow on one instrument, from the label to the decision	a case in the lecture
then	trees and ensembles, for numbers and for labels	the same workflow

Every model from here on gives a probability. What changes is how the probability is produced; the threshold, the costs and the scores stay the same.

Before the next session **CASE**

- **Exercises:** rare classes, thresholds and costs, `class_weight`, calibration buckets, three-class labels and their scores, and k-NN against logistic regression.
- **Case Part 9:** the risk report gets a threshold, a cost, and a check on its probabilities.

Stuck? Ask a friend, ask an AI for a hint rather than an answer, or email jobo@econ.au.dk.